VIDALSTORE EP2 · CHECKLIST MAESTRO, CONCEPTOS AMQP Y METODOLOGÍA D9

Asignatura: DSY1107 - Desarrollo Cloud Native I · **Institución:** DUOC UC · **Evaluación:** EP2 (Mensajería Asíncrona, Clúster RabbitMQ, DLQ y Persistencia)
Equipo de Desarrollo: David (Infraestructura / Plataforma), Oscar Garrido (Microservicio Administrador / QA), Iván (Worker Eventos / Consumidores)
Estado Global: 209 de 209 tests unitarios e integración pasando (100% de éxito) en los 6 repositorios oficiales.

1. Resumen Ejecutivo del Estado del Sistema y Auditorías Automatizadas

El proyecto VidalStore cuenta con la cadena sincrónica de 4 capas aprobada (EP1), topología AMQP declarativa centralizada, control de flujo por prefetch(1), Quorum Queues con Raft en RabbitMQ 4 y persistencia relacional en PostgreSQL 18 con TypeORM. A continuación se consolida el estado verificado tras la auditoría técnica integral:

Área de Auditoría	Indicadores EP2	Estado Verificado	Veredicto y Próxima Acción Técnica
Cadena Sincrónica (EP1)	IE1, IE2, IE3, IE4, IE5	**100% LOGRADO**	Gateway (:8080), BFF (:3001), Angular (:4200) y sessionStorage validados. 144 commits listos.
Clúster RabbitMQ 2 Nodos	IE9, IE10, IE11	**100% OPERATIVO**	classic_config y Quorum Queues activas. Desacoplado depends_on en plataforma y raíz. Peer discovery automático.
Microservicio Admin (:3020)	IE6, IE7, IE8, IE17, IE18	**100% COMPLETADO**	89 tests Vitest pasando. Mapeo semántico traducir() y oyente canal.on('error') plenamente integrados en dev.
Consumidores y Cartas Muertas	IE12, IE13, IE15, IE16	**100% FUNCIONAL**	prefetch(1), reintentos con pausa, nack(false, false) y consumidor de dlq.reprocesar 100% integrados.
Políticas de Retención (IE19)	IE19	**100% INTEGRADO**	3 políticas versionadas en rabbitmq/politicas.json (drop-head y reject-publish) con script de importación automatizado.

2. Checklist Técnico Detallado y Tareas por Integrante

Bloque 1 · Infraestructura y Clúster RabbitMQ (vidalstore-plataforma · David)

ID	Rama / Componente	Tarea Técnica / Archivo Afectado	Estado	Criterio de Aceptación y Veredicto
INF-1	`feature/cluster-autodiscovery-politicas`	Políticas de retención y desborde	**LISTO / EN DEV**	Versionado en rabbitmq/politicas.json con drop-head (avisos/trabajo), reject-publish (DLQ), TTL y script de importación.
INF-2	`feature/cluster-autodiscovery-politicas`	Desacoplar depends_on en docker-compose.yml	**LISTO / EN DEV**	Quitado 'condition: service_healthy' en rabbitmq2; ambos nodos arrancan en paralelo y sincronizan vía cluster.conf.
INF-3	`feature/quorum-queues-cluster`	Adoptar Quorum Queues en shared/topologia.ts	**EN DEV**	Configurado {'x-queue-type': 'quorum'} en las 6 colas. /api/queues reporta estado 'minority' ante caída de rabbit2.
INF-4	`feature/variables-entorno-cluster`	Completar variables en .env.example	**EN DEV**	Declarado RABBITMQ_ERLANG_COOKIE, RABBITMQ_DEFAULT_USER y credenciales de PostgreSQL.
INF-5	`feature/diagnostico-quorum-critico`	Validar comando de Quórum Crítico (Código 69)	**LISTO**	'rabbitmq-diagnostics check_if_node_is_quorum_critical' retorna código 69 advirtiendo pérdida de quórum.

Bloque 2 · Microservicio Administrador (vidalstore-admin · Oscar Garrido)

ID	Rama / Componente	Tarea Técnica / Archivo Afectado	Estado	Criterio de Aceptación y Veredicto
ADM-1	`feature/error-handling-traducir`	Mapeo semántico de errores con traducir(err)	EN DEV	Mapea NOT_FOUND a 404, PRECONDITION_FAILED a 409, y timeout/minority a 503 Service Unavailable.
ADM-2	`feature/canal-efimero-resiliente`	Oyente de error en canal efímero	EN DEV	Registrado 'canal.on("error", ...)' de inmediato para evitar que Node.js termine por errores de canal.
ADM-3	`feature/validacion-nombres-regex`	Validar nombres con regex en crear-cola.dto.ts	EN DEV	Añadido @Matches(/^[a-zA-Z0-9._-]+$/) para impedir inyecciones de caracteres ilegales en URLs.
ADM-4	`feature/pruebas-curl-administrador`	Batería de 8 pruebas cURL lista para defensa	LISTO	Ejecutar en vivo: colas (201/400/409/404), exchanges, bindings, métricas y reproceso DLQ.
ADM-5	`feature/ep-dlq-reproceso`	Reprocesamiento DLQ idempotente en PostgreSQL	EN DEV	Valida UUID v4, rechaza 400 si ya fue reprocesado, republica en dlq.reprocesar y actualiza BD. 89 tests OK.

Bloque 3 · Worker de Eventos y Consumidores (vidalstore-eventos · Iván)

ID	Rama / Componente	Tarea Técnica / Archivo Afectado	Estado	Criterio de Aceptación y Veredicto
EVT-1	`feature/calibracion-auditoria-l7`	Manejo de Idempotencia y Código 23505	EN DEV	Captura código 23505 en procesar-evento-auditoria.ts y nack sin requeue en reintentos.ts. 22 tests OK.
EVT-2	`feature/prueba-mensaje-envenenado`	Guion de prueba Mensaje Envenenado	LISTO	Publicar JSON corrupto: worker rechaza con nack(false, false) y viaja a DLQ para persistir en mensajes_muertos.
EVT-3	`feature/consumer-reproceso-dlq`	Consumidor de dlq.reprocesar en eventos	EN DEV	Implementado en reproceso-dlq.consumidor.ts: escucha dlq.reprocesar, reinyecta al flujo operativo y persiste.

Batería de Pruebas Automatizadas del Ecosistema (209 Tests Aprobados)

Repositorio / Servicio	Tecnología / Framework	Archivos de Test	Tests Pasando	Cobertura Crítica Evaluada
`vidalstore-admin`	Vitest / NestJS	7 suites	**89 / 89 (100%)**	traducir(), Zero Trust, canal.on('error'), DTOs forbidNonWhitelisted, Reproceso DLQ.
`vidalstore-backend`	Jest / NestJS	19 suites	**72 / 72 (100%)**	AuthGuard, ScopesGuard, GroupsGuard (Salida A), compras, catálogo, amqplib publish.
`vidalstore-eventos`	Vitest / TypeScript	7 suites	**22 / 22 (100%)**	prefetch(1), SQLSTATE 23505, nack(false, false), backoff exponencial, reproceso DLQ.
`vidalstore-gateway`	Jest / NestJS	4 suites	**19 / 19 (100%)**	Proxy reverso perimetral, validación tokens RS256, cabeceras X-Forwarded.
`vidalstore-frontend`	Karma / Jasmine	5 suites	**6 / 6 (100%)**	Amplify Hosted UI, sessionStorage, tokenInterceptor Bearer, catálogo interactivo.
`vidalstore-plataforma`	Vitest / Docker Compose	1 suite	**1 / 1 (100%)**	Sintaxis YAML compose, healthcheck Postgres y classic_config de RabbitMQ.

3. Conceptos Clave de Mensajería AMQP (D7, D8, D9 y Laboratorios L6-L8A)

En la arquitectura de VidalStore, el productor **nunca publica directo en una cola**, sino en un **Exchange** que enruta vía **Bindings**. A continuación se presentan las definiciones técnicas y su aplicación práctica:

Tipo Exchange	Exchange VidalStore	Mecanismo de Enrutamiento	Propósito y Comportamiento en Solución
topic	`vidalstore.eventos`	Patrones jerárquicos delimitados por puntos con comodines * y #.	Difunde hechos de negocio. compra.* llega a Avisos y Correos; # captura 100% en Auditoría.
direct	`vidalstore.comandos` `vidalstore.dlx`	Coincidencia exacta punto a punto: Routing Key === Binding Key.	Envía órdenes directas (correo.enviar) o desvía cartas muertas a su DLQ homónima (ej: dlq.avisos).
fanout	*(Reservado)*	Difusión masiva (broadcast) a todas las colas enlazadas.	Ignora la routing key. Ideal para invalidar cachés distribuidas o alertas generales a todos los nodos.
headers	*(Reservado)*	Enrutamiento basado en pares clave/valor del header (x-match: all/any).	No depende de la routing key textual. Útil para enrutamiento por metadatos (geolocalización, tenant).

Reglas de Enlace, Comodines y las 3 Durabilidades Indispensables

Concepto Clave	Definición Técnica en AMQP	Implementación Concreta en VidalStore EP2
Binding	Regla de unión entre un Exchange y una Cola. Si no hay binding que calce, el mensaje se descarta.	`canal.bindQueue(COLAS.avisos, EXCHANGES.eventos, 'compra.*')`
Comodín *	Sustituye **exactamente una palabra** delimitada por puntos.	compra.* calza con compra.realizada y compra.cancelada, pero NO con sub-niveles.
Comodín #	Sustituye **cero, una o múltiples palabras**.	Enlazado a q.auditoria: garantiza el registro inmutable de cualquier evento emitido en el sistema.
Las 3 Durabilidades (D8 Slides 22-24)	Para sobrevivir reinicios sin pérdida: 1) Volumen nombrado, 2) Cola durable, 3) Mensaje persistent.	Volumen rabbitmq1-data + { durable: true } en assertQueue + { persistent: true } en publish.
Prefetch(1)	Limita mensajes sin confirmar por consumidor a 1, evitando colapso de RAM ante lentitud de Postgres.	await this.canal.prefetch(1) ejecutado inmediatamente tras crear el canal en mensajeria.service.ts.
Quorum Queues	Colas replicadas basadas en quórum Raft (■N/2■ + 1). Reemplazan colas espejo obsoletas.	Declaradas con arguments: { 'x-queue-type': 'quorum' } en las 3 colas de trabajo y las 3 DLQ.

4. Metodología Oficial de Diagnóstico en 4 Pasos (D9) y las 4 Salidas del Consumidor

La clase D9 exige desterrar el ensayo y error: diagnosticar es **reducir el espacio de lo posible mediante evidencia**:

Paso 1: Síntoma	Paso 2: Hipótesis	Paso 3: Comando	Paso 4: Cambio
Qué se ve con palabras exactas (ECONNREFUSED :5432), nunca 'no funciona'. Log se lee **de arriba hacia abajo**. En stack trace, mirar la **1ra línea de src/**.	Explicación técnica concreta y **falsable**: *'Postgres aún no acepta conexiones porque initdb sigue corriendo en el arranque inicial'*.	Comando que confirma o descarta: docker compose ps. **Si descarta, se vuelve al Paso 2**; jamás se pasa al Paso 4 a ciegas.	Una sola modificación, la que la hipótesis confirmada señala. Se vuelve a correr para verificar si el síntoma desapareció.

Salida	Condición Técnica en VidalStore	Acción AMQP	Destino Final
1. Éxito	Evento persistido con éxito en PostgreSQL.	`canal.ack(mensaje) tras INSERT`	Fuera de la cola (OK)
2. Duplicado	Reentrega choca con UNIQUE (evento_id) (SQLSTATE **23505**).	`Log WARN + canal.ack(mensaje)`	Fuera de la cola (sin duplicar ni DLQ)
3. Permanente	JSON corrupto, sin usuarioSub o error SQL 22.../23...	`canal.nack(mensaje, false, false)`	DLX $ ightarrow$ mensajes_muertos (text)
4. Transitorio	Postgres no contesta (timeout, socket cerrado, ECONNREFUSED).	`conReintentos (2s, 4s); si falla, nack(false, false)`	Reintenta; si persiste, DLQ

5. Catálogo Forense de Errores para la Defensa (Explicados con los 4 Pasos)

Caso / Error	1. Síntoma Exacto	2. Hipótesis & 3. Comando	4. Cambio & Argumento
1. Postgres no listo	connect ECONNREFUSED :5432 al arrancar worker.	H: Postgres está en initdb. C: docker compose ps muestra starting.	Configurar healthcheck con pg_isready y condition: service_healthy.
2. Colas perdidas	Tras recrear RabbitMQ, list_queues sale vacío.	H: Nodename Erlang cambió por ID aleatorio. C: eval 'node().' da rabbit@id.	Fijar hostname: rabbitmq1 en Compose para anclar el directorio Mnesia/Khepri.
3. Unacked congelado	Ready sube, Unacked clavado en 1 y Deliver en 0.	H: Consumidor no hizo ack. C: rabbitmqctl list_queues ...	Agregar canal.ack(msg) al terminar; prefetch(1) bloqueaba la entrega.
4. Bucle veneno	Miles de errores por segundo de JSON roto en log.	H: Consumidor hizo requeue: true. C: docker compose logs -f	Usar canal.nack(msg, false, false) hacia DLX para aislar el mensaje.
5. Duplicado 23505	violates unique constraint UQ_evento_id	H: Reentrega at-least-once. C: SELECT count(*) WHERE evento_id=... da 1.	Capturar código 23505 y dar ack; la base ya está protegida por UNIQUE.
6. Error 406	406 PRECONDITION_FAILED for queue 'q.avisos'	H: Cola existía con argumentos distintos. C: rabbitmqctl list_queues arguments	Borrar cola previa con delete_queue y redeclarar con argumentos nuevos.
7. Estado minority	Quorum queues no confirman al caer rabbit2.	H: Mayoría ■2/2■+1=2 rota. C: /api/queues muestra state: minority.	2 nodos toleran 0 caídas (Raft). Explicar que HA real exige 3 nodos impares.
8. Falla semántica 503	Admin responde 503 en vez de error 500 ciego.	H: Broker caído o sin quórum. C: Llamar endpoint admin sin broker.	Método traducir() mapea excepciones AMQP a códigos HTTP limpios.
9. Auth Postgres	password authentication failed for user	H: Clave del .env cambió pero volumen persistió. C: Comparar hash volumen vs .env.	Restaurar clave del volumen o resetear con docker compose down -v.
10. BOLA/IDOR	Usuario compra o consulta a nombre de un tercero.	H: Controlador lee usuarioSub del body. C: Enviar usuarioSub arbitrario.	Regla de Oro: extraer siempre identidad de req.user.sub (token validado).

6. Protocolo de Defensa en Vivo (Los 15 Minutos de D6) y las 5 Ventanas Obligatorias

Ventana	Ubicación / URL	Vista / Comando Preparado	Propósito en la Evaluación
1. Host Docker	Terminal plataforma	`docker compose ps listo para failover`	Demostrar contenedores sanos y ejecutar 'docker stop vidalstore-rabbitmq2'.
2. RabbitMQ UI	`http://localhost:15672` `http://localhost:15673`	Pestañas Overview, Exchanges y Queues	Mostrar clúster de 2 nodos, bindings topic/direct, colas Quorum y estado minority.
3. Logs Worker	Terminal eventos	`docker compose logs -f eventos`	Evidenciar recepción de eventos en tiempo real, ack, rechazos sin requeue y persistencia.
4. Swagger Admin	`http://localhost:3020/api/docs`	Endpoints CRUD y /dlq/:id/reproceso	Demostrar creación de colas, rechazo 400 forbidNonWhitelisted y reprocesamiento forense.
5. Frontend Web	`http://localhost:4200`	Angular DevTools (Network y Application)	Demostrar compra interactiva, tokens en sessionStorage y revocación en vivo.

REGLAS ESTRICTAS DE GIT (D6 Y POLÍTICA DEL PROYECTO):
• Todos los repositorios se encuentran en la rama **dev** con **209 de 209 tests aprobados (100%)**.
• **NO USAR COMANDOS DE GIT SIN AUTORIZACIÓN EXPRESA.** Commits redactados en español y modo imperativo con prefijos semánticos ('feat:', 'fix:', 'test:', 'docs:', 'chore:').
• Las ramas feature son efímeras y deben eliminarse tras el merge a **dev**. Antes de evaluar, fusionar **dev** a **main** de forma atómica y declarar el hash en AVA.